16-03-2026

Elastic File System(EFS):

EFS = a fully managed, elastic, shared file system that **uses the NFS** (Network File System) protocol.
It gives you a shared folder in the cloud that can be mounted on many EC2 instances at the same time.

Elastic means you don't pre-size it — it automatically grows/shrinks as you add or remove files.

NFS = Network File System (Linux/Unix file sharing standard).
With EFS:

- AWS runs the file server infrastructure.
- You don't patch OS, maintain storage arrays, or worry about failover.
- You just create a file system and mount it via NFS (e.g., mount -t nfs4 ...).

Supports NFSv4.1 (default) and NFSv4.0.

Elastic — No Capacity Planning

- You don't pick a size. It grows **from MBs to petabytes** as you add data.
- You only **pay for what you store**.
- Automatically shrinks when you delete files (after a short delay).
- No need to resize like EBS.

Performance & Storage Classes

- Two performance modes:
- *General Purpose* (default) – low latency, great for most apps.
- *Max I/O* – better for very large, highly parallel workloads.
- Two storage classes:
- *Standard* – frequently accessed files.
- *Infrequent Access (IA)* – cheaper for cold files; automatic lifecycle policy can move unused files.

Use Cases

- **Shared web content:** images, scripts, uploads used by many servers.

- **CMS / WordPress cluster:** all nodes mount the same wp-content/uploads.
- **Home directories:** shared user home directories in a Linux fleet.
- **Big data analytics:** shared data lake for multiple compute nodes.
- **Dev/test:** quickly share build artifacts or logs.

Pricing:

- Pay per GB stored per month (Standard or IA rate).
- Pay for data transfer between AZs if applicable.
- No minimum capacity; truly pay-as-you-use.

POSIX semantics: good for Linux apps; not native for Windows (use FSx for Windows File Server instead).

Amazon FSx for Windows File Server:

Windows equivalent of EFS

Provides a **native SMB (Server Message Block)** file share — the same protocol Windows uses for shared folders
If **EFS** is like a Linux NFS share, **FSx for Windows File Server** is a **Windows shared drive** that you map with a drive letter (e.g., Z:).
Key Features

- **Native Windows file system (NTFS):**
- Supports NTFS permissions, Access Control Lists (ACLs), file attributes.
- Works with Windows tools (Explorer, PowerShell).
- **Fully managed:**
- AWS handles servers, storage, patching, hardware failures.
- **Highly available:**
- Runs in **Multi-AZ** (optional single-AZ).
- **Elastic capacity & performance:**
- Scale **storage** (up to petabytes) and **throughput**.
- **Active Directory integration:**
- Join to your on-prem or AWS Managed AD.

- Use existing **users/groups/permissions**.
- **Backups:**
- Automatic daily backups, point-in-time restore.

Simple rule to teach:

- **Linux apps → EFS**
- **Windows apps → FSx for Windows File Server**

S3:

apps actually **store and fetch** objects from S3.

Bucket = your cloud folder (globally unique name).
Object = a file (photo, PDF, video).
Key = the path/name inside the bucket (events/diwali/IMG_001.jpg).
Access = controlled by permissions; apps talk to S3 via **API**.

"**Do I see snapshots in my S3 bucket?**" → EBS snapshots are stored *by AWS* on S3 **internally** (you won't see them in your bucket).
S3 is **object storage**, not a disk

Object Storage (S3)

- What it is:
- Stores **objects** = file + metadata + unique key in a flat namespace (buckets).
- **No file system / folders** internally (folders are a UI illusion).
- Access via **HTTP API / URLs**, not as a block device.
  each file has a unique URL.
- How you use it:
- Upload with console, CLI, or API (PUT object).
- Download via HTTPS (GET object).

- Set permissions, versioning, lifecycle policies.
- AWS example:

  S3 bucket hosting website images:
  https://**mybucket**.s3.amazonaws.com/photos/img1.jpg

  Apps integrate with S3 by **API/SDK** — not by mounting.

Bucket:

Buckets are — a **top-level container** for objects

- **Globally unique** name (can't reuse someone else's).
- Tied to a **region** (data lives in that AWS region).
- Can hold **unlimited objects**.

Rules to mention:

- Name must be globally unique (my-app-logs-2025).
- Lowercase letters, numbers, hyphens only.
- Region matters for latency & compliance.

Objects

- Each **file you upload** becomes an **object**.
- **Object = data (file) + metadata + key (unique path).**
- You can think of the key like a **full file path** inside the bucket.
- There's **no true folder hierarchy** — folders in the console are a *visual trick* (just prefixes in the key).

```
my-bucket
  └── photos/
      └── vacation/img1.jpg
```

Key here = photos/vacation/img1.jpg

Metadata examples:

- Content-Type (image/jpeg),
- Size,
- Last modified date,

Object URLs

- Every object can be accessed via an **HTTP(S) URL**:
  https://my-bucket.s3.amazonaws.com/photos/vacation/img1.jpg
- By default, **private** (needs IAM/temporary signed URL).
- You can make public or generate **pre-signed URLs** for temporary sharing.

"S3 = buckets + objects. Buckets are globally unique containers; objects are files with metadata and keys. Apps use HTTPS/SDKs to put and get objects."

Regions
Each bucket is created **in one AWS Region** (e.g., ap-south-1 = Mumbai, us-east-1 = N. Virginia).

- **Data physically stored** in that region.
- **Latency**: closer to users → faster.
- **Cost**: transfer between regions costs extra.
- **Compliance**: some data must stay in-country.

URL Format:
https://<bucket>.s3.<region>.amazonaws.com/<key>

Moving buckets between regions is **not supported** (must recreate & copy objects).

S3 Storage Classes

- S3 isn't "one size fits all."
- Some data is accessed every day (hot), some rarely (cold).
- AWS gives multiple classes so you pay less for data you rarely touch.

Durability stays 11 9's for almost all classes
availability drops a little for IA/One Zone.
S3 Intelligent-Tiering
Use Intelligent-Tiering if you're unsure of access patterns.

- **Goal:** If you **don't know** how often files will be accessed.
- AWS monitors access:
- **Frequent Access tier** (Standard-like cost).
- **Infrequent Access tier** (Standard-IA cost).
- Automatic move between tiers after **30 days of no access**.
- *Optional deeper archive tiers* (Archive / Deep Archive) inside Intelligent-Tiering for older data.
- Small **monitoring fee per object** but saves if usage patterns vary.

Lifecycle Configuration (Storage Management)

- Automate **moving, archiving, or deleting objects** to save cost.
- Policy is applied at **bucket or prefix** level.

🔹 **Actions You Can Configure**

1. **Transition Actions**

- Day 0 → Upload to **Standard**
- After 30 days → Move to **Standard-IA**
- After 180 days → Move to **Glacier Deep Archive**.
- Move object to cheaper class after X days.
- Example:

2. **Expiration Actions**

- Permanently delete object after X days.
- Good for logs or temporary files.

3. **Noncurrent Version Actions**

- If versioning is enabled: transition or expire **old versions** after X days.

Move data automatically from Standard → IA → Glacier when it gets old.

Rules are **evaluated once per day** (not real-time).

Real-life example: Company logs — hot for 1 month, rarely touched after.

S3 Versioning

- **Versioning** lets a bucket keep **multiple versions of the same object**.
- Every time you **upload a file with the same key**, S3 **creates a new version** instead of overwriting.
- Helps recover from **accidental delete or overwrite**.

Default: OFF (no versions kept).

Key Behaviors

- When enabled, **new uploads** to the same key get a **new Version ID**.
- If you **delete** an object, S3 doesn't erase it; it adds a **delete marker**.
- You can **restore** older versions or **permanently delete** specific ones.

Delete File

- S3 creates a **Delete Marker** (you can see older versions under **"Show versions"**).

Restore

- Delete the **Delete Marker** → file is visible again.
- Or download an older version directly.

Delete action (console or DELETE API)

- Adds a **Delete Marker** → the object disappears from the "current view", but older versions still exist.

To truly erase:

- Remove the **Delete Marker** (if you want to restore) or
- Delete **each version ID** you want gone.

Impt:

- **Delete Marker ≠ permanent deletion**; it just hides.
- If bucket has **MFA Delete** enabled, you need an **MFA token** for permanent deletion.
- **Lifecycle rules** can automatically purge old versions after X days (good for cost control).

If You want to **completely remove** the object from your bucket —
meaning: delete both the **delete marker** and **all previous versions**.

MFA Delete

- Adds an **extra layer of protection** for versioned buckets.
- To **permanently delete** a version or **suspend versioning**, you need **MFA (multi-factor authentication)**.
  Why: Prevents accidental or malicious deletes.
  "What happens if a developer accidentally deletes a bucket with production logs?"
- Versioning = **Recoverable**; MFA Delete = **Extra safety**.
  Versioning costs more (because old versions count toward storage).

S3 Event Notifications

- Often you want to **react when something happens in a bucket**:
- New file arrives → process it.
- File deleted → update database.
- Object restored from Glacier → notify user.
- Instead of **polling S3**, you can configure **event notifications**.
  Polling = your application **repeatedly calls S3** on a schedule to check
  (e.g., ListObjects, HeadObject):
  - Has any new file arrived?
  - Did a file get updated or deleted?

Events That Can Trigger:

- **s3:ObjectCreated:*** – when new object is uploaded (PUT, POST, COPY, or multipart upload complete).
- **s3:ObjectRemoved:*** – when object is deleted.
- **s3:ObjectRestore:*** – when Glacier object is restored.
- **s3:Replication:*** – replication related events.

Where Notifications Can Go

1. **Amazon SNS (Simple Notification Service)**

- Sends **push notifications / emails / messages** to subscribers.
- Example: *"Send email to admin whenever new file uploaded."*

2. **Amazon SQS (Simple Queue Service)**

- Puts message in a **queue** for further processing.
- Example: *"Microservice picks up each message and processes file."*

3. **AWS Lambda**

- Trigger serverless function immediately.
- Example: *"Image uploaded → Lambda resizes to thumbnails."*

Demo (Console)

1. Go to Bucket → Properties → Event Notifications → Create Event Notification.
2. Name: ImageUploadTrigger.
3. Event Types: choose All object create events.
4. Destination:

- If using Lambda: pick an existing function (e.g., resizeImage).
- If using SNS: select topic (e.g., new-uploads-topic).
- If using SQS: select queue.

S3 Pricing:
Storage (GB per month)

- Main driver of cost – how many GB you store × price of chosen storage class.

Requests & Retrievals

- You pay per API request (PUT, GET, LIST, COPY, DELETE).

Data Transfer

- Inbound (uploading) → FREE.
- Outbound to the internet → charged per GB after free tier (first 1 GB/mon free).
- Transfer within same region AWS services → mostly free.
- Cross-region replication → you pay for outgoing data.

Replication Rule in S3?
Replication rules define **how and which objects** in a source S3 bucket are automatically **copied (replicated)** to another bucket (destination).

Replication ensures that data is **safely and automatically duplicated** to another bucket — often in **another AWS Region** or **same Region** for backup, compliance, or latency reasons.

What is Cross-Region Replication?

- **Cross-Region Replication (CRR)** is an S3 feature that **automatically copies objects** from one bucket (source) in a region to another bucket (destination) in a **different AWS region**.
- Used for **disaster recovery, latency reduction, and compliance**.

When Does It Actually Get Copied?

- Usually, replication happens **within seconds or a few minutes**.
- But it **is not immediate** — there's a **slight delay** depending on:
- The number of new objects,
- Network traffic between regions,
- S3's internal replication queues.

So, the copy happens **"soon after"** — but **not guaranteed instantaneously**.

Key Points

- **Source bucket** must have **versioning enabled**.
- **Destination bucket** must be in **another region** (for CRR; if in same region it's SRR — same-region replication).
- **IAM role** is created by AWS to perform the copy.
- **New and updated objects** are replicated automatically.
- **Existing objects** are **not copied** unless you enable **replication for existing objects** or manually trigger a batch copy.

How to Set Up (console overview)

1. Enable **Versioning** on both source and destination buckets.
2. On the **source bucket** → **Management** → **Replication rules** → **Add rule**.
3. Select:

- **Destination bucket & region**,
- Optionally filter (prefix/tags),

- Optionally enable **Replication Time Control**.

4. AWS creates an **IAM role** to handle replication.

S3 Transfer Acceleration?

- **Amazon S3 Transfer Acceleration (TA)** is a feature that speeds up **uploads and downloads** of objects to S3 **over long distances**.
- It uses **Amazon CloudFront's globally distributed edge locations** as **accelerated entry points**.
- Data travels over Amazon's **high-speed private backbone network** instead of the slower public internet

How It Works

1. **You enable TA on a bucket.**
2. AWS gives you a special URL:

   https://<bucketname>.s3-accelerate.amazonaws.com

3. When you upload/download using this URL:

- Your client connects to the **nearest AWS edge location**.
- Data travels through AWS's **private backbone** to the S3 bucket's region.
- Faster than routing packets across random public ISPs.

How to Enable (Console)

1. Go to **S3** → **Bucket** → **Properties** → **Transfer acceleration**.
2. Switch to **Enabled** (or **Enabled with performance hints**).
3. Use the **accelerate endpoint**:
   https://mybucket.s3-accelerate.amazonaws.com

Real App Architecture:
Goal: Combine S3, EBS, EFS in one picture.

- Scenario: Web app on EC2.
- EBS: OS + app binaries + database files.
- EFS: Shared uploads/media between servers.
- S3: Static content (images, backups).

What Is an ACL (Access Control List)?
Access Control List (ACL) is an *old-style* access control mechanism in Amazon S3 that defines **who can access your bucket or objects** and **what actions they can perform** (like read or write).
It's one of the **two ways** to manage permissions in S3:

1. **Bucket Policies / IAM Policies** → modern, recommended way
2. **ACLs (Access Control Lists)** → older, low-level way (still supported for backward compatibility)

What is Server Access Logging in S3?
Server Access Logging is a feature that records **detailed information about every request** made to your S3 bucket.
In simple terms — it keeps a **log file of "who accessed what, when, and how."**
This helps you **track usage, monitor security, and analyze traffic patterns.**
Notes:

- S3 Has Infinite Capacity
- You pay **only for what you actually store**

What Is Multipart Upload?
Multipart Upload is a method that lets you **upload a single large object as smaller parts**, in parallel, and then combine them in S3.
You can upload files in **parts** (chunks) — each part is between **5 MB and 5 GB** in size.
AS S3 stores each part temporarily,
and when all parts are uploaded,
S3 **reassembles** them into one complete object
Upload failures can resume by re-sending only failed parts

You can track upload progress & abort incomplete uploads

You **cannot** upload one *single file* larger than **5 TB**.
But you **can** upload **millions of smaller files** totaling **hundreds of TB or PB** in the same bucket

What is a Checksum? à Data Integrity
A **checksum** is a short, unique digital signature (a "fingerprint") of a file —
used to **verify that data wasn't changed, corrupted, or tampered with** during upload, storage, or download.

In simple words:
A checksum ensures that **"what you upload to S3 is exactly what's stored and later downloaded."**

What Is Object Lock: Object Lock is an S3 feature that lets you **prevent objects from being deleted or overwritten for a fixed time period or permanently**.

It's a way of making data **"write-once, read-many" (WORM)** — meaning:
Once written → can't be modified or deleted
Until the lock expires (or forever, if you choose permanent retention)

What is *Requester Pays* in S3?

Requester Pays is an S3 bucket feature where the **person downloading or accessing the data (the requester)** pays for the **data transfer and request costs**, instead of the **bucket owner**.
In normal S3 usage:

- The **bucket owner** pays for all costs —
(storage + data transfer + requests).
But if **Requester Pays** is enabled:
- The **requester** pays the **request and data transfer** costs.
- The **bucket owner** still pays for **storage**.

What is Static Website Hosting in S3?
Static website hosting in S3 means using an **S3 bucket** to host a **website made of static files** — like HTML, CSS, JavaScript, and images — **without any servers (EC2 or backend code).**
AWS S3 acts as a **simple web server** for files that **don't change dynamically**.

https://devdocs.io/

What is Metadata in S3?
Metadata in Amazon S3 is information that describes the object — it's data about the data.
It doesn't change the object's content, but provides extra details that help AWS or your application understand how to handle that object.
Who created it:- ***When it was last modified, ***What type of file it is,* How it should be cached or downloaded.

ASSIGNMENTS : What is **API?**